

LAB # 13

SQL to MongoDB Mapping Chart

OBJECTIVE:

- SQL To Mongodb Mapping Chart

MongoDB is a cross-platform, document oriented database that provides, high performance, high availability, and easy scalability. MongoDB works on concept of collection and document.

Database

Database is a physical container for collections. Each database gets its own set of files on the file system. A single MongoDB server typically has multiple databases.

Collection

Collection is a group of MongoDB documents. It is the equivalent of an RDBMS table. A collection exists within a single database. Collections do not enforce a schema. Documents within a collection can have different fields. Typically, all documents in a collection are of similar or related purpose.

Document

A document is a set of key-value pairs. Documents have dynamic schema. Dynamic schema means that documents in the same collection do not need to have the same set of fields or structure, and common fields in a collection's documents may hold different types of data.

Terminology and Concepts

The following table presents the various SQL terminology and concepts and the corresponding MongoDB terminology and concepts.

SQL	MONGODB
database	database
Table	collection
Row	document or BSON document

column	field
Index	index
primary key	primary key
aggregation (e.g. group by)	aggregation pipeline

Create and Alter

The following table presents the various SQL statements related to table-level actions and the corresponding MongoDB statements.

SQL Schema Statements	MongoDB Schema Statements
<pre>CREATE TABLE people (id MEDIUMINT NOT NULL AUTO_INCREMENT, user_id Varchar(30), age Number, status char(1), PRIMARY KEY (id)</pre>	Implicitly created on first insertOne() or insertMany() operation. The primary key <code>_id</code> is automatically added if <code>_id</code> field is <pre>db.people.insertOne({ user_id: "abc123", status: "A" })</pre>
<pre>ALTER TABLE people ADD join_date DATETIME</pre>	Collections do not describe or enforce the structure of its documents; i.e. there is no structural alteration at the collection level. However, at the document level, updateMany() operations can add fields to existing documents using the \$set operator. <pre>db.people.updateMany({ }, { \$set: { join_date: new Date() } })</pre>
—	Collections do not describe or enforce the structure of its documents; i.e. there is no structural alteration at the collection level. However, at the document level, updateMany() operations can remove fields from documents using the \$unset operator.

	<pre>db.people.updateMany({ }, { \$unset: { "join_date": "" } })</pre>
<code>DROP TABLE people</code>	<code>db.people.drop()</code>

Insert

The following table presents the various SQL statements related to inserting records into tables and the corresponding MongoDB statements.

SQL INSERT Statements	MongoDB insertOne() Statements
<pre>INSERT INTO people(user_id, age, status) VALUES ("bcd001", 45, "A")</pre>	<pre>db.people.insertOne({ user_id: "bcd001", age: 45, status: "A" })</pre>

Select:

SQL SELECT Statements	MongoDB find() Statements
<pre>SELECT * FROM people</pre>	<code>db.people.find()</code>
<pre>SELECT id, user_id, Status FROM people</pre>	<pre>db.people.find({ }, { user_id: 1, status: 1 })</pre>
<pre>SELECT * FROM people WHERE status = "A"</pre>	<pre>db.people.find({ status: "A" })</pre>
<pre>SELECT user_id, status FROM people WHERE status = "A"</pre>	<pre>db.people.find({ status: "A" }, { user_id: 1, status: 1, id: 0 })</pre>

	)
<pre>SELECT * FROM people WHERE age > 25</pre>	<pre>db.people.find({ age: { \$gt: 25 } })</pre>
<pre>SELECT * FROM people WHERE user_id like "bc%"</pre>	<pre>db.people.find({ user_id: /bc/ })</pre>
<pre>SELECT * FROM people WHERE status = "A" ORDER BY user_id DESC</pre>	<pre>db.people.find({ status: "A" }).sort({ user_id: -1 })</pre>

Update Records

The following table presents the various SQL statements related to updating existing records in tables and the corresponding MongoDB statements.

SQL Update Statements	MongoDB updateMany() Statements
<pre>UPDATE people SET status = "C" WHERE age > 25</pre>	<pre>db.people.updateMany({ age: { \$gt: 25 } }, { \$set: { status: "C" } })</pre>
<pre>UPDATE people SET age = age + 3 WHERE status = "A"</pre>	<pre>db.people.updateMany({ status: "A" }, { \$inc: { age: 3 } })</pre>

Delete Records

The following table presents the various SQL statements related to deleting records from tables and the corresponding MongoDB statements.

SQL Delete Statements	MongoDB deleteMany() Statements
<pre>DELETE FROM people WHERE status = "D"</pre>	<pre>db.people.deleteMany({ status: "D" })</pre>
<pre>DELETE FROM people</pre>	<pre>db.people.deleteMany({})</pre>

Lab Tasks:

Create a comparison chart of mongoDB and SQL queries and perform

- Create and alter queries
- Insert,update,delete queries

on resturant collection